

# **Plato: The Planning Ontology**

Specification v1.0.0

Trier University  
Department IV - Computer Science  
Chair of Business Informatics II

German Research Center for Artificial Intelligence  
Trier Branch  
Experience-based Learning Systems

David Jilg  
david.jilg@dfki.de

# Contents

|          |                                  |           |
|----------|----------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>              | <b>1</b>  |
| <b>2</b> | <b>Motivation</b>                | <b>1</b>  |
| <b>3</b> | <b>Specification</b>             | <b>1</b>  |
| 3.1      | Core . . . . .                   | 2         |
| 3.2      | Types . . . . .                  | 3         |
| 3.3      | Entities . . . . .               | 5         |
| 3.4      | Parameters . . . . .             | 5         |
| 3.5      | Entity Order . . . . .           | 5         |
| 3.6      | AWS . . . . .                    | 5         |
| 3.7      | Advanced Statements . . . . .    | 9         |
| 3.7.1    | Disjunctive Statements . . . . . | 9         |
| 3.7.2    | Conjunctive Statements . . . . . | 9         |
| 3.7.3    | Implications . . . . .           | 11        |
| 3.7.4    | Universal Statements . . . . .   | 11        |
| 3.7.5    | Existential Statements . . . . . | 11        |
| 3.8      | Cost . . . . .                   | 11        |
| 3.8.1    | Target Function . . . . .        | 13        |
| 3.8.2    | Cost Equations . . . . .         | 13        |
| 3.9      | Metrics . . . . .                | 16        |
| 3.10     | Numeric Constants . . . . .      | 16        |
| <b>4</b> | <b>Available Tooling</b>         | <b>17</b> |
| <b>5</b> | <b>Appendix</b>                  | <b>18</b> |
|          | <b>References</b>                | <b>20</b> |

# Figures

|      |                                                                         |    |
|------|-------------------------------------------------------------------------|----|
| 3.1  | A visual representation of the Plato scheme . . . . .                   | 2  |
| 3.2  | The core of Plato . . . . .                                             | 3  |
| 3.3  | The type system . . . . .                                               | 3  |
| 3.4  | The entity concept . . . . .                                            | 4  |
| 3.5  | The parameter concept . . . . .                                         | 6  |
| 3.6  | The entity order concept . . . . .                                      | 7  |
| 3.7  | The atomic world state concept . . . . .                                | 8  |
| 3.8  | Advanced statements . . . . .                                           | 10 |
| 3.9  | Service costs . . . . .                                                 | 12 |
| 3.10 | Target functions for service costs . . . . .                            | 13 |
| 3.11 | The cost equation concept . . . . .                                     | 14 |
| 3.12 | A binary expression tree expressing the formula $(A * B) + C$ . . . . . | 14 |
| 3.13 | Parameter referencing in cost equations . . . . .                       | 15 |
| 3.14 | Cost metrics . . . . .                                                  | 16 |
| 3.15 | Numeric constants . . . . .                                             | 17 |

# Tables

|     |                                                         |    |
|-----|---------------------------------------------------------|----|
| 5.1 | All classes of the Plato scheme . . . . .               | 18 |
| 5.2 | All object properties of the Plato scheme . . . . .     | 18 |
| 5.3 | All data properties of the Plato scheme . . . . .       | 18 |
| 5.4 | All annotation properties of the Plato scheme . . . . . | 19 |
| 5.5 | All individuals of the Plato scheme . . . . .           | 19 |

# 1. Introduction

This document describes Plato, the Planning Ontology. Plato is an ontology schema for the semantic description of planning environments and is intended to support the use of AI planning. Plato is open source (MIT License). All documents describing it, as well as references to the official tooling that supports it, are available on GitHub<sup>1</sup>.

This document is structured as follows. Section 2 outlines the motivation for creating Plato. Section 3 then describes the Plato specification. Section 4 presents the tooling that already supports Plato. Finally, Section 5 provides a comprehensive list of all concepts used in the Plato schema.

## 2. Motivation

In current AI planning practice, semantic information about the planning environment is usually modeled directly in a planning domain using languages such as the Planning Domain Definition Language (PDDL). Modeling such planning domains is time-consuming and difficult. The same applies to changes in the planning environment that later need to be reflected in the domain. As a result, planning domains are difficult to maintain.

Another drawback of directly modeling semantic information in a planning domain is that the resulting model is often tied to a specific planning system. Even when using a generic modeling language such as PDDL, the available choice of planning systems is restricted because most PDDL planners support only a subset of the standard.

To address these limitations, Malburg et al. [3] present an approach that enables the automatic and dynamic transformation of semantic information about a planning environment into PDDL planning domains. In this approach, the semantic information is not stored in a static planning domain but in an ontology. This simplifies maintenance and also enables the generation of different planning domains for the specific planning system being targeted. To evaluate the approach, a prototypical implementation is developed that transforms a semantic description of a Fischertechnik simulation factory from the FTOnto ontology [1] into a PDDL planning domain. Since the FTOnto is only viable for Fischertechnik simulation factories, we developed the Plato ontology. Plato is intended to support the semantic modeling of generic planning environments for use with the approach presented by Malburg et al. and in other use cases.

## 3. Specification

---

<sup>1</sup><https://github.com/DavidJilg/Plato>

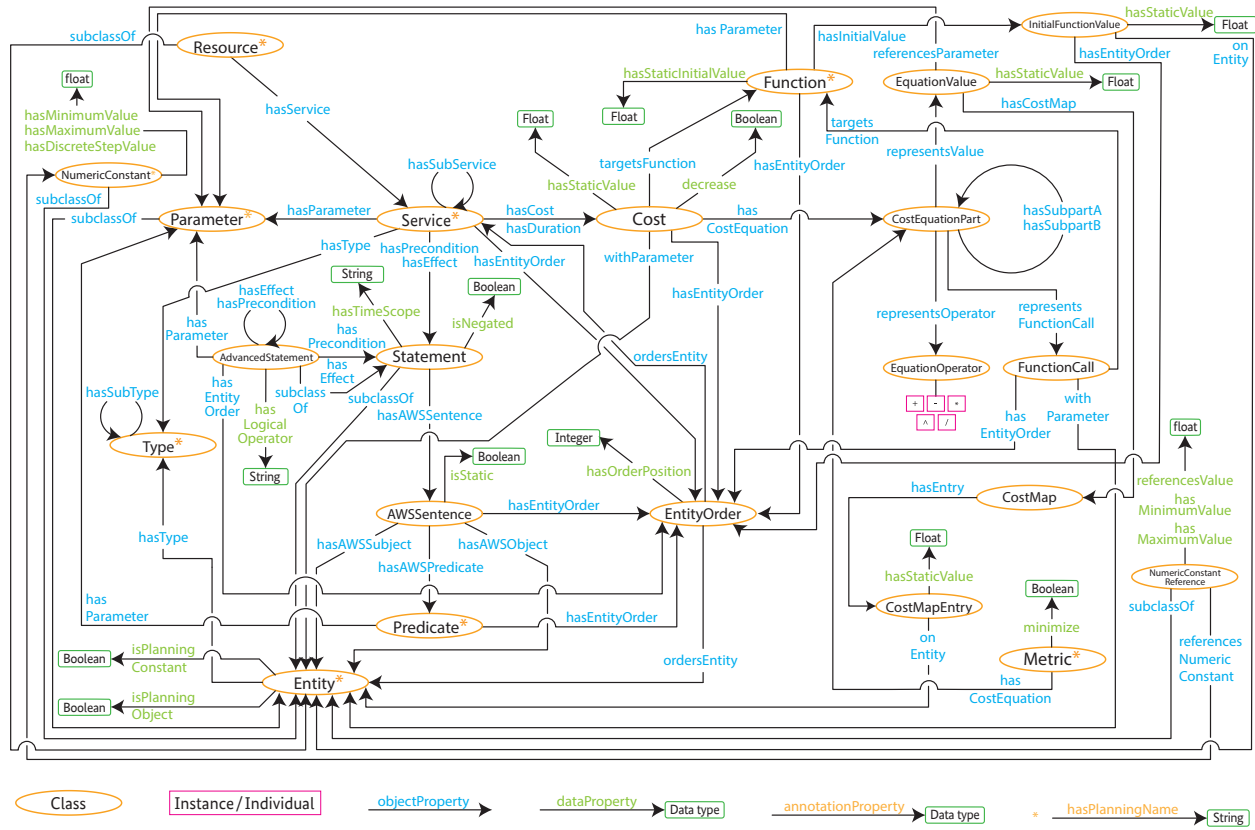


Figure 3.1: A visual representation of the Plato scheme. Due to space constraints, inverse object properties and some details are omitted.

This section introduces the concepts of the Plato schema and describes each part. Each part is accompanied by a graphical representation that serves as a visual aid. Figure 3.1 provides an overview of the whole ontology and a legend that is valid for all figures in this document. These figures always include the previously explained parts, but those parts are grayed out. It should also be noted that, due to size constraints, the visual representations do not contain all information required to model an ontology using the Plato schema. For example, object properties are shown only in one direction, which means that inverse properties are omitted.

The Plato ontology schema is modeled using the Web Ontology Language (OWL). It provides a set of classes, object properties, data properties, annotation properties, and named individuals that can be used to describe a planning domain. Figure 3.1 shows an overview of Plato. For each ontology modeled with Plato the schema version has to be specified using the ‘plato:version’ annotation property to annotate the ontology. Currently 1.0.0 is the only available version, but this might change in the future. The version is important to ensure that the ontology can be correctly interpreted by tools that support Plato, even if the schema evolves over time.

### 3.1 Core

The core of Plato consists of the ‘Service’ class, which represents a step in the processes that are to be planned or, more generally, an action in a planning domain. Figure 3.2 shows this part of the Plato schema. For example, when converting Plato to a PDDL planning domain, each instance of this class can be transformed into a PDDL action.

A service can have multiple sub-services, which are referenced through the ‘hasSubService’ object property. This information can be used for hierarchical planning. If a service has more than one

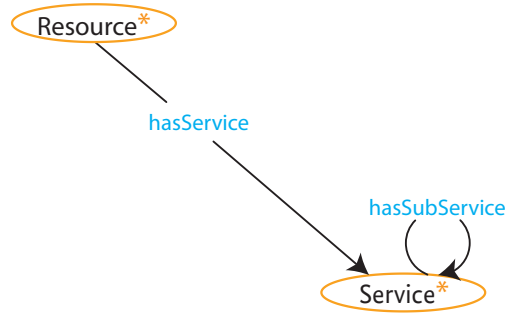


Figure 3.2: The core of Plato

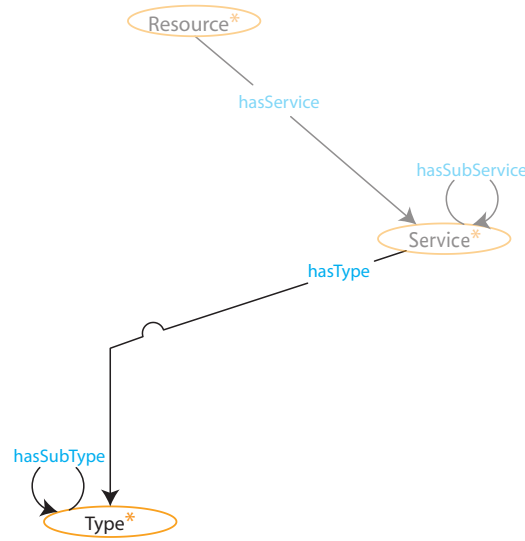


Figure 3.3: The type system

sub-service, the sub-services need to be ordered using the `EntityOrder` concept described in Section 3.5. Since services are usually provided by resources in the planning environment, the Plato `Resource` class can be used to represent a resource that provides a service. The `hasService` and `isProvidedBy` object properties are used to express this information from the perspectives of the resource and the service, respectively.

Whenever inverse object properties exist, both sides of the relationship must be asserted consistently. For example, when a machine references a service through the `hasService` property, the service must reference the machine using the inverse property `isProvidedBy`. Section 5 provides a complete list of the available object properties and their inverse properties.

Both services and resources should have a planning name specified through the annotation property `hasPlanningName`. This name is used when generating planning domains from the ontology.

## 3.2 Types

For transformations into planning domain languages such as PDDL that support types, Plato contains a type system that allows types to be assigned to various concepts, such as objects, constants and services. Figure 3.3 shows this part of the Plato schema. The type system uses a hierarchy composed of individuals of the `Type` class. Each type individual should have a planning name. In addition, it can have an arbitrary number of subtypes specified through the `hasSubType` property. Multiple inheritance and cyclic inheritance patterns are not allowed.

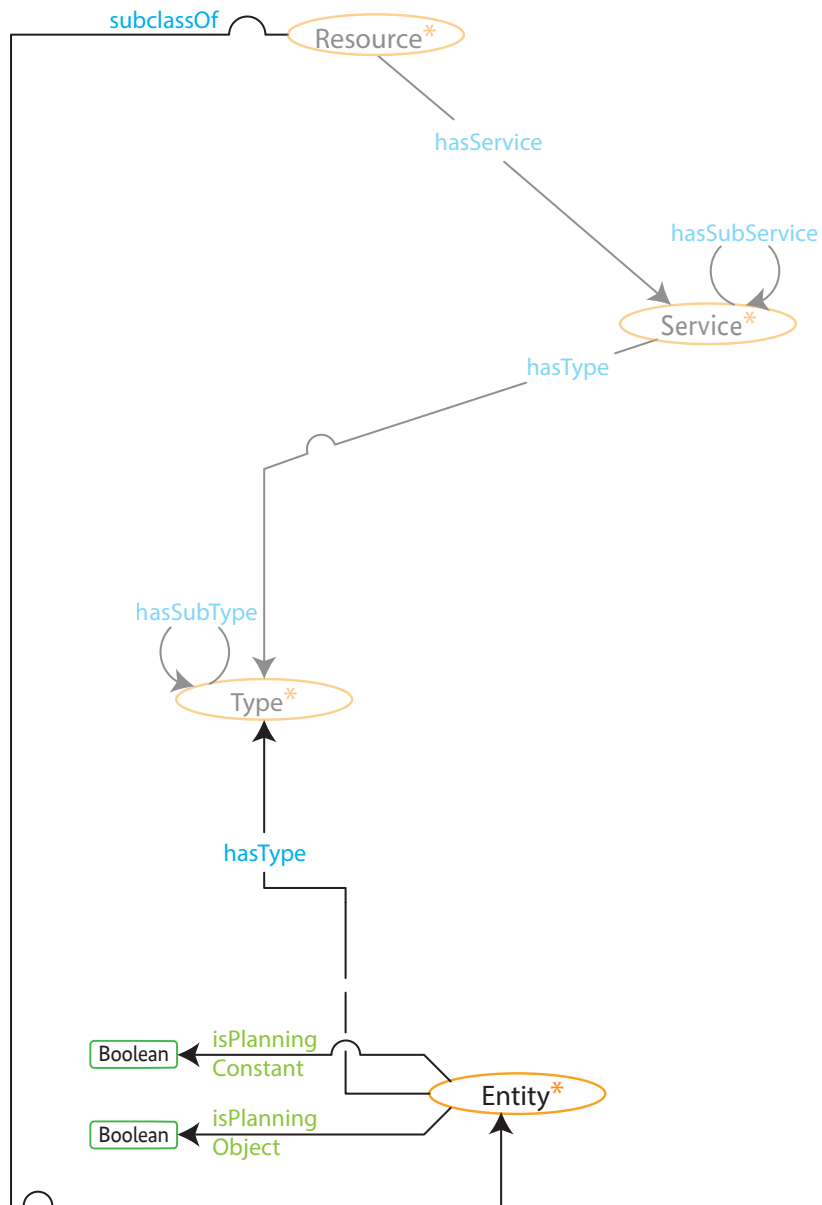


Figure 3.4: The entity concept

### 3.3 Entities

During the development of Plato, many classes introduced to represent entities in the schema were found to be structurally very similar. One design goal was therefore to keep the ontology schema as simple as possible while still allowing all necessary information to be represented. This is important because semantic structures such as ontologies can become cluttered when too many classes and properties are introduced. For this reason, most classes representing entities are consolidated into a single class called ‘Entity’. Figure 3.4 shows this part of the Plato schema.

This design significantly reduces the complexity of the schema by decreasing the number of concepts and by reducing the number of distinct property domains and ranges, since these characteristics also apply to subclasses. For example, the ‘Resource’ class is a subclass of ‘Entity’ and can therefore use all data and object properties defined for ‘Entity’.

Each individual associated with the ‘Entity’ class, or with one of its subclasses, can have a type and a planning name. In addition, it can have data properties indicating whether it is a planning object or a planning constant. These data properties are used to distinguish between individuals that should exist in the generated planning domain and individuals that are used only to describe a particular planning problem.

### 3.4 Parameters

Several concepts in a planning domain require parameters. The most obvious example is the previously introduced ‘Service’ class. A service such as drilling a hole needs to be configurable through parameters such as the hole size.

For this purpose, Plato contains the ‘Parameter’ class. This class can be used together with the ‘hasParameter’ object property to denote that an individual, such as a service, has parameters. Figure 3.5 shows this part of the Plato schema. Each parameter must have a planning name and an associated type that specifies which values can be provided for it. If an individual has more than one parameter, the parameters must be ordered using the ‘EntityOrder’ concept described in the following section.

### 3.5 Entity Order

Many individuals in an ontology based on the Plato schema have associated individuals that need to be ordered. For example, a service can have multiple parameters or multiple sub-services that need to appear in a specific order. For this purpose, Plato contains the ‘EntityOrder’ class. Figure 3.6 shows this part of the Plato schema.

For each association that needs to be ordered, there must be an individual of the ‘EntityOrder’ class that provides the corresponding order position. For example, for every parameter associated with a service, the service must also be linked to an ‘EntityOrder’ individual using the ‘hasEntityOrder’ object property. The ‘EntityOrder’ individual must specify an order position as an integer value using the ‘hasOrderPosition’ data property. It must also point to the entity being ordered through the ‘ordersEntity’ object property.

There is one exception to the rule that an ‘EntityOrder’ only orders individuals associated with the ‘Entity’ class. This exception concerns the sub-services of a service. Since sub-services also need to be ordered, the domain of the ‘ordersEntity’ object property additionally includes the ‘Service’ class.

### 3.6 AWS

When modeling a planning domain, it must be possible to specify under which conditions services can be executed and which effects they have on the world. For this purpose, Plato extends the Atomic World State (AWS) concept introduced by Klein et al. in the FTOnto ontology [1]. Figure 3.7 shows this part of the Plato schema.

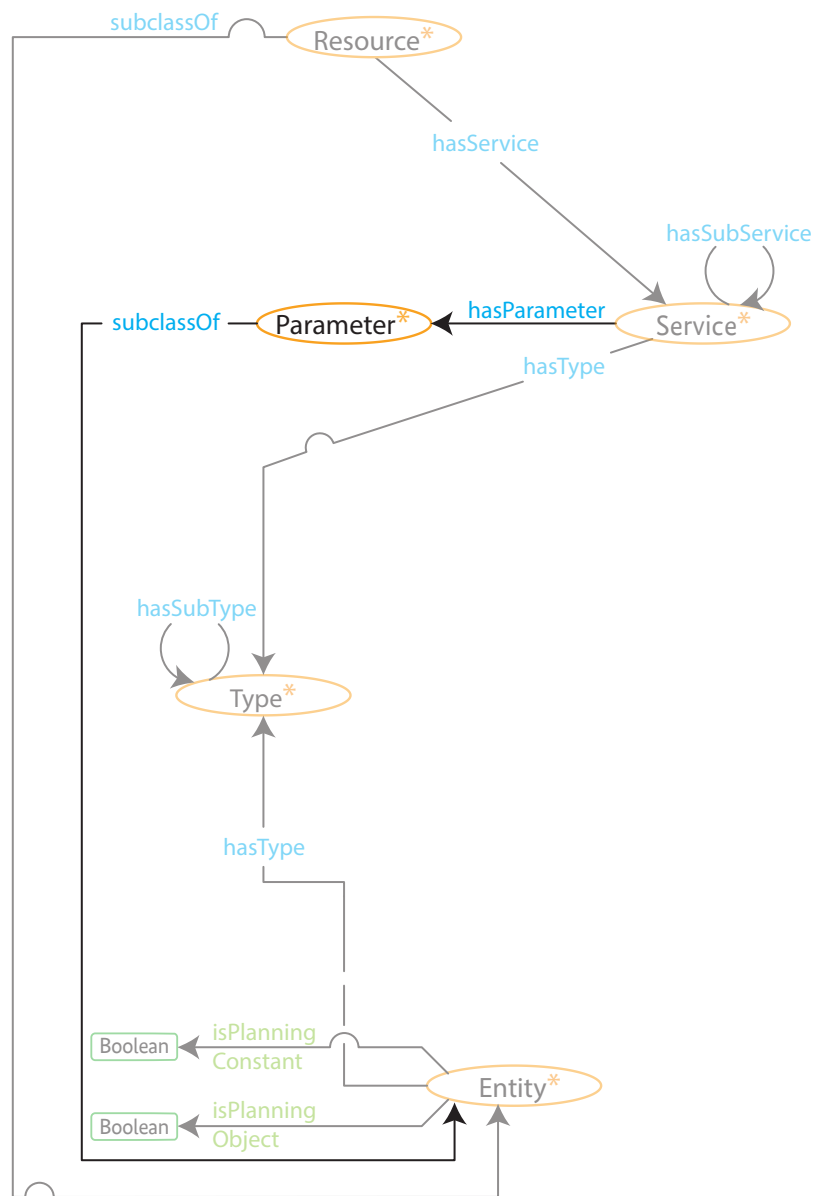


Figure 3.5: The parameter concept



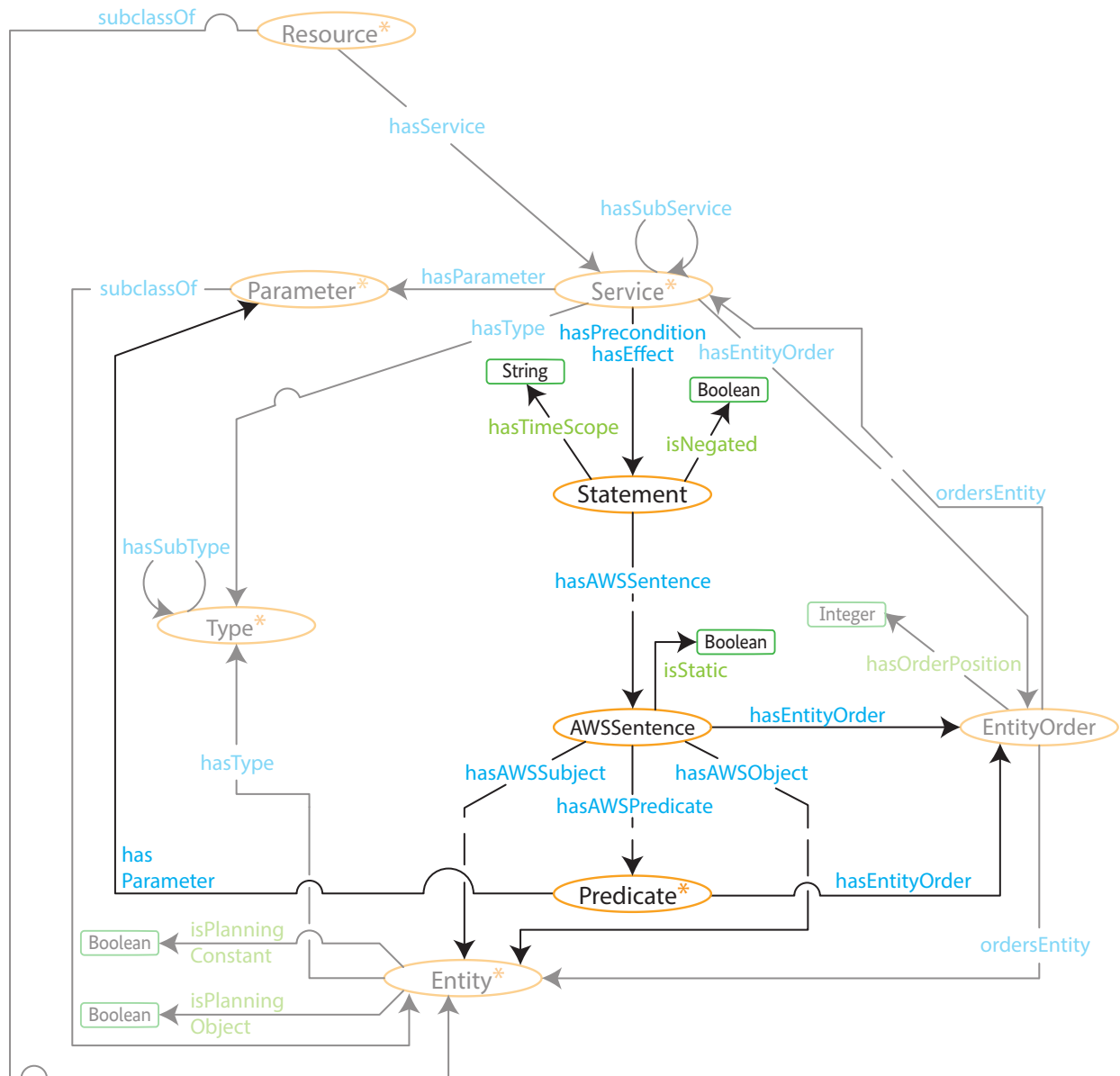


Figure 3.7: The atomic world state concept

An ‘AWSSentence’ must have exactly one subject and exactly one predicate associated with it through the ‘hasAWSSubject’ and ‘hasAWSPredicate’ object properties. Optionally, it can have an arbitrary number of objects associated with it through the ‘hasAWSObject’ object property. If more than one object is present, the objects must be ordered using the ‘EntityOrder’ concept introduced in Section 3.5.

Each predicate in the planning domain must have a planning name associated with it through the ‘hasPlanningName’ annotation property. In addition, it can have an arbitrary number of parameters specified through the ‘hasParameter’ object property. If more than one parameter is present, the parameters must be ordered using the ‘EntityOrder’ concept introduced in Section 3.5.

In addition to the ‘AWSSentence’ instances used in preconditions and effects, static sentences can also be specified to represent facts that do not change in the planning environment. A static ‘AWSSentence’ must not be associated with any precondition or effect, and it must have the ‘isStatic’ data property set to true. If this data property is not present, the ‘AWSSentence’ is considered non-static.

## 3.7 Advanced Statements

In addition to basic statements, Plato supports advanced logical statements such as conjunctive, disjunctive, implicative, universal, and existential combinations of other statements. For this purpose, the Plato schema contains the ‘AdvancedStatement’ class. An individual of this class can be referenced by a service through the ‘hasPrecondition’ object property to model an advanced precondition or through the ‘hasEffect’ object property to model an advanced effect.

Like basic statements, advanced statements can be negated using the ‘isNegated’ data property and assigned a time scope using the ‘hasTimeScope’ data property. The ‘AdvancedStatement’ class allows the definition of disjunctive statements, conjunctive statements, implications, universal statements, and existential statements. When an advanced statement is used as a precondition, all of these logical operators can be applied. When it is used as an effect, conjunctive, implicative, universal, and existential effects can be modeled. Disjunctive effects are not supported.

An advanced statement can itself reference other statements. These referenced statements can be linked through either the ‘hasPrecondition’ or the ‘hasEffect’ object property. For nested statements, the choice between these two properties does not determine whether the advanced statement is used as a precondition or as an effect. The role of an advanced statement is determined only by the property through which the advanced statement itself is linked to a service. This makes advanced statements easier to reuse because the same logical expression can be referenced as a precondition in one service and as an effect in another service without changing its internal structure.

### 3.7.1 Disjunctive Statements

Disjunctive statements are nested statements that are joined using the logical ‘or’ operator. To create a disjunctive statement, an individual of the ‘AdvancedStatement’ class must be created and linked to other statements through either the ‘hasPrecondition’ or the ‘hasEffect’ object property. The referenced statements can be either basic statements or advanced statements. The logical operator must be specified using the ‘hasLogicalOperator’ data property with the value ‘or’.

Disjunctive statements may be used as preconditions. They must not be used as effects, since Plato does not support disjunctive effects.

### 3.7.2 Conjunctive Statements

Conjunctive statements are nested statements that are joined using the logical ‘and’ operator. To create a conjunctive statement, an individual of the ‘AdvancedStatement’ class must be created and linked to other statements through either the ‘hasPrecondition’ or the ‘hasEffect’ object property. The referenced statements can be either basic statements or advanced statements. The logical operator must be specified using the ‘hasLogicalOperator’ data property with the value ‘and’.

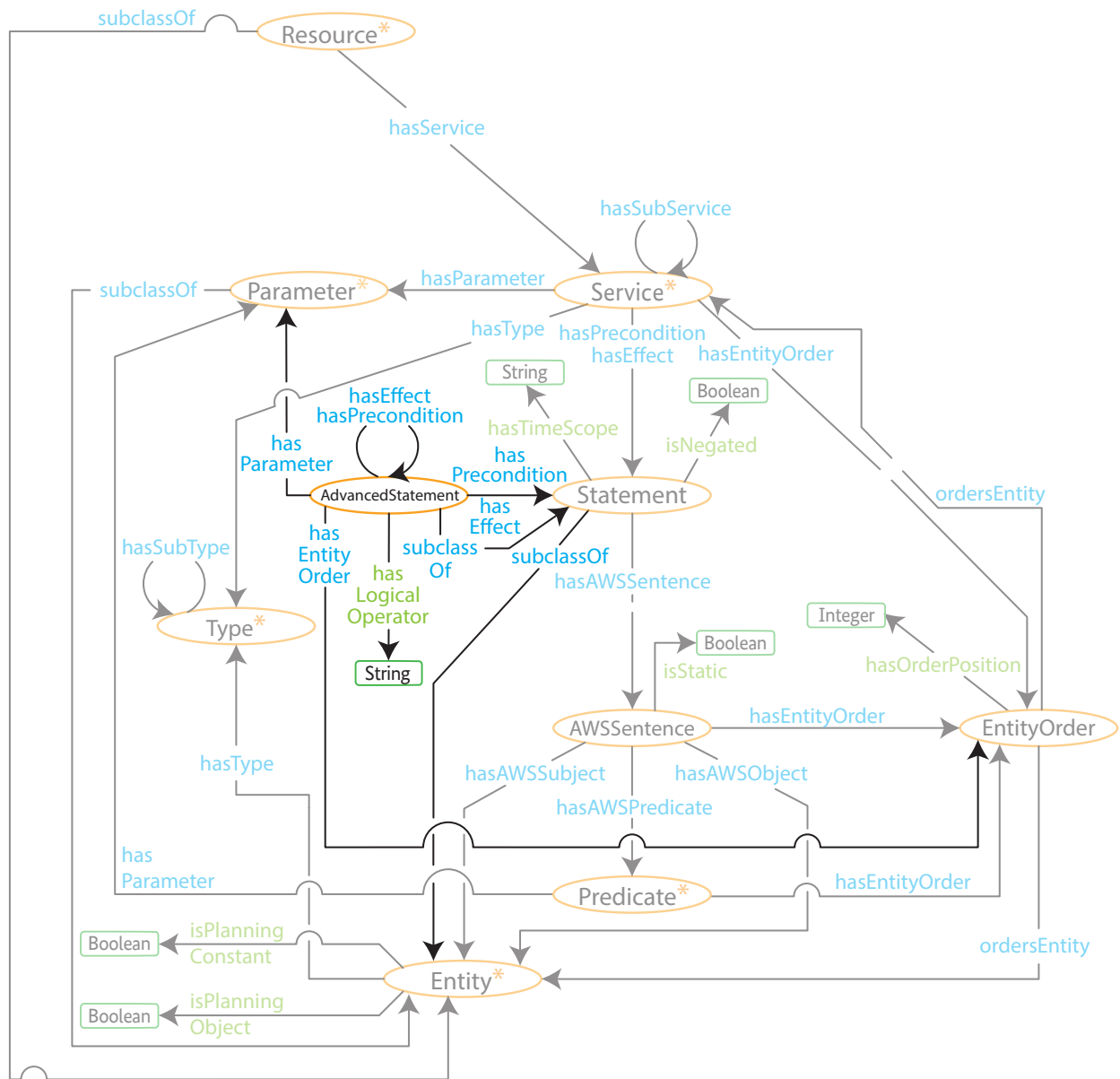


Figure 3.8: Advanced statements

### 3.7.3 Implications

Implications consist of two statements, where the first statement implies the second. To create an implication, an individual of the ‘AdvancedStatement’ class must be created and linked to exactly two other statements through either the ‘hasPrecondition’ or the ‘hasEffect’ object property. The referenced statements can be either basic statements or advanced statements.

The two statements must be ordered using the ‘EntityOrder’ concept introduced in Section 3.5. In addition, the logical operator must be specified using the ‘hasLogicalOperator’ data property with the value ‘imply’.

### 3.7.4 Universal Statements

A universal statement requires a logical statement to hold for all objects of a certain type. To create a universal statement, an individual of the ‘AdvancedStatement’ class must be created and linked to one statement through either the ‘hasPrecondition’ or the ‘hasEffect’ object property.

To specify which objects should be used in the universal statement, a parameter with an appropriate type must be added through the ‘hasParameter’ object property. All objects of that type are then tested against the referenced statement. The parameter must also appear in the referenced statement. Finally, the logical operator must be specified using the ‘hasLogicalOperator’ data property with the value ‘forall’.

### 3.7.5 Existential Statements

An existential statement requires that there exists at least one object of a certain type that satisfies a logical statement. To create an existential statement, an individual of the ‘AdvancedStatement’ class must be created and linked to one statement through either the ‘hasPrecondition’ or the ‘hasEffect’ object property.

To specify which objects may satisfy the statement, a parameter with an appropriate type must be added through the ‘hasParameter’ object property. All objects of that type are then tested against the referenced statement. The parameter must also appear in the referenced statement. Finally, the logical operator must be specified using the ‘hasLogicalOperator’ data property with the value ‘exists’.

## 3.8 Cost

In most planning scenarios, finding any valid solution is not the only goal. In many cases, the objective is also to find a solution that is as optimal as possible. For this reason, it must be possible to specify service costs so that the chosen planning approach can optimize with respect to them. Plato therefore contains an expressive cost system. Figure 3.9 shows the core of this system.

The core of the cost system consists of the ‘Cost’ class. Each service can have an arbitrary number of cost individuals associated with it through the ‘hasCost’ or ‘hasDuration’ object properties. The duration of a service is treated as a special kind of cost. In contrast to normal costs, however, each service may have only one ‘hasDuration’ object property assertion.

A cost individual can have two optional data property assertions. With the ‘hasStaticValue’ property, a static cost value can be specified. Non-static costs can be represented using the cost equation concept introduced in Section 3.8.2. With the ‘decrease’ data property, it can be specified that the cost value should be negative.

In Plato, costs are mapped to cost functions, which are explained in the following section. If the ‘decrease’ property is set to true, the value of the corresponding cost function is decreased rather than increased.

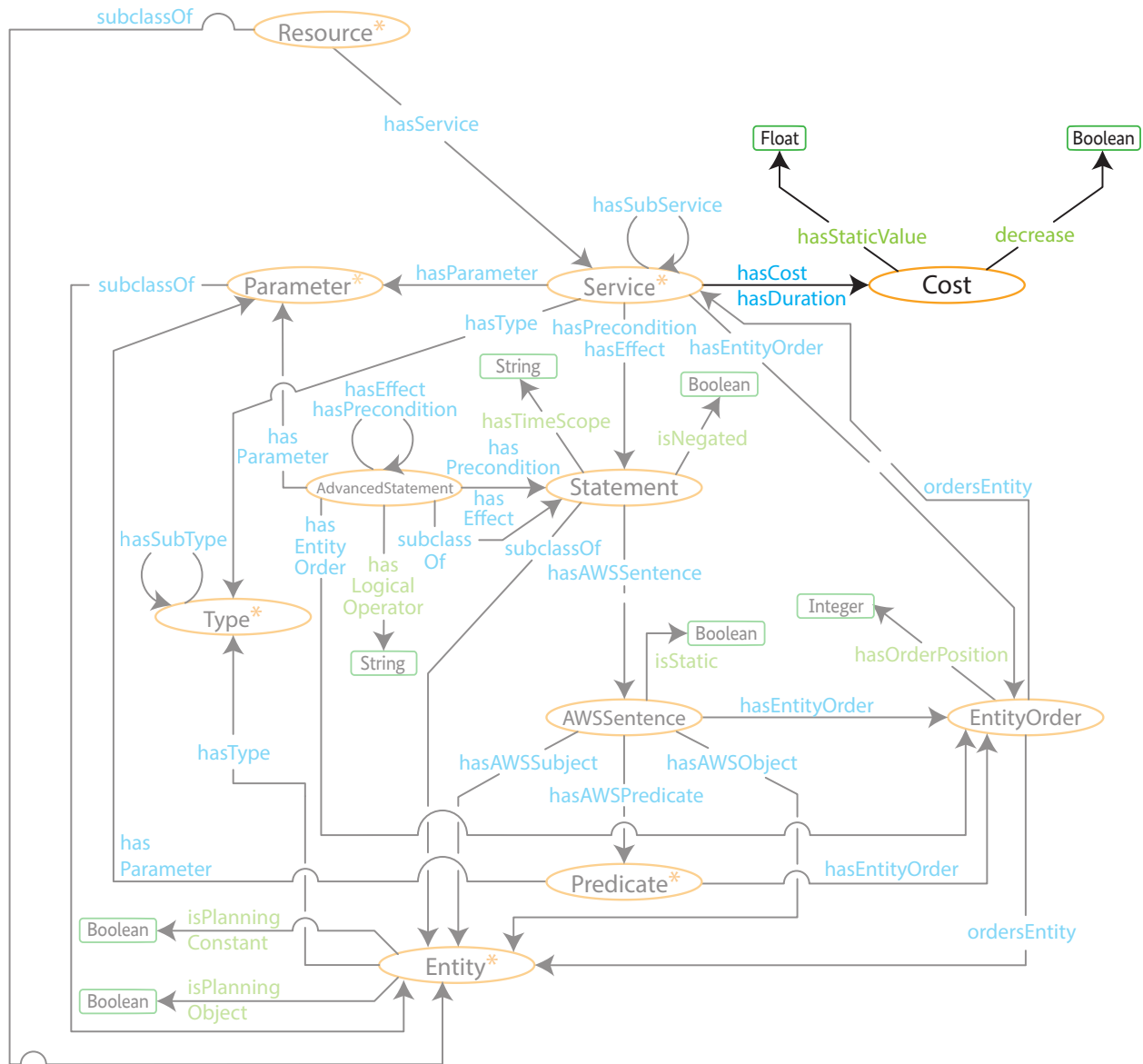


Figure 3.9: Service costs

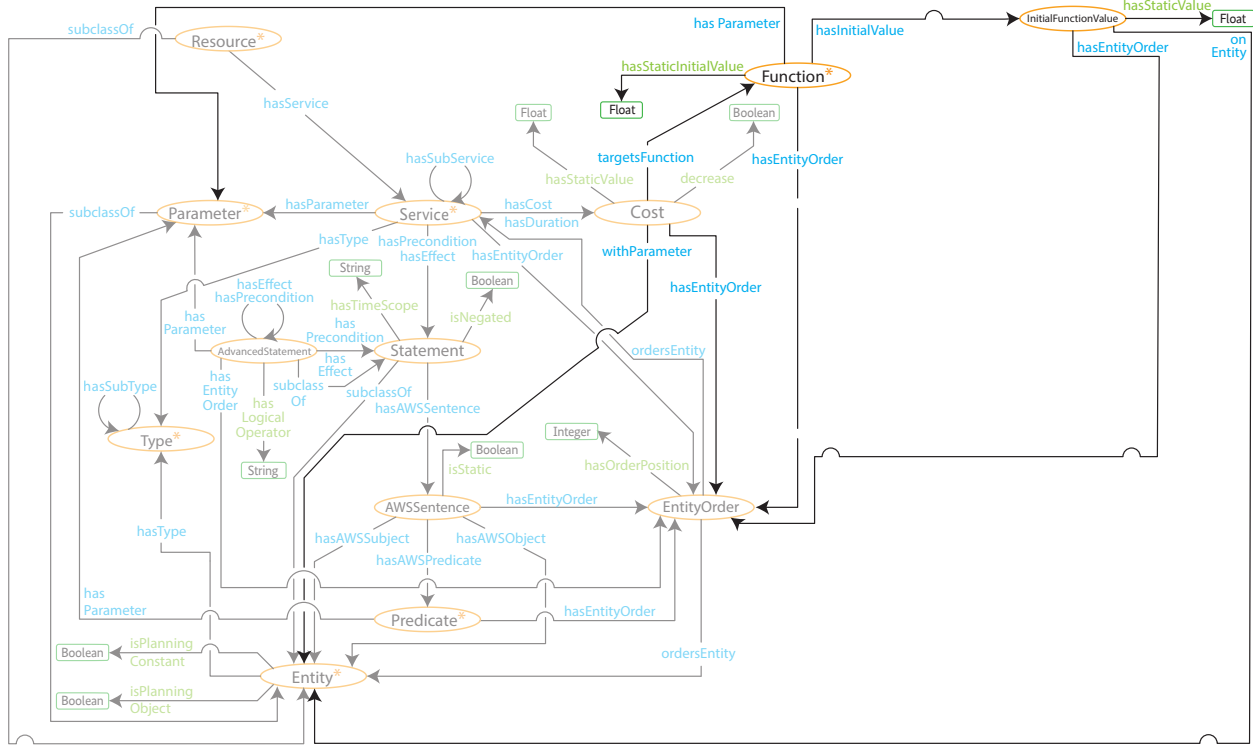


Figure 3.10: Target functions for service costs

### 3.8.1 Target Function

As mentioned above, all costs specified for services in Plato must be mapped to cost functions. Figure 3.10 shows this part of the Plato schema.

To map a cost to a cost function, the ‘targetsFunction’ object property is used. If the function requires parameters, it must also be specified which parameter values are passed to the function when the cost is applied. This is done through the ‘withParameter’ object property. If more than one parameter value is required, the parameters must be ordered using the ‘EntityOrder’ concept introduced in Section 3.5.

Cost functions are represented using the ‘Function’ class. This class can also be used to define functions that are not related to costs. Each function must have a planning name. An initial value can also be specified for each function. If no initial value is given, a value of 0 is assumed. If the function has no parameters, the initial value can be specified through the ‘hasStaticInitialValue’ data property. If the function has parameters, initial values for specific parameter combinations can be defined through the ‘InitialFunctionValue’ class and the ‘hasInitialValue’ object property.

An ‘InitialFunctionValue’ individual must provide a float value through the ‘hasStaticValue’ data property. It must also specify an ordered set of entities that act as the parameter values for which the initial value applies. These parameter values are defined through the ‘onEntity’ object property together with the ‘EntityOrder’ concept introduced in Section 3.5.

### 3.8.2 Cost Equations

The cost equation concept of Plato makes it possible to define non-static service costs. Figure 3.11 shows this part of the Plato schema.

The representation of cost equations is based on binary expression trees<sup>2</sup>. Figure 3.12 visualizes the concept of binary expression trees.

<sup>2</sup>[https://en.wikipedia.org/wiki/Binary\\_expression\\_tree](https://en.wikipedia.org/wiki/Binary_expression_tree)

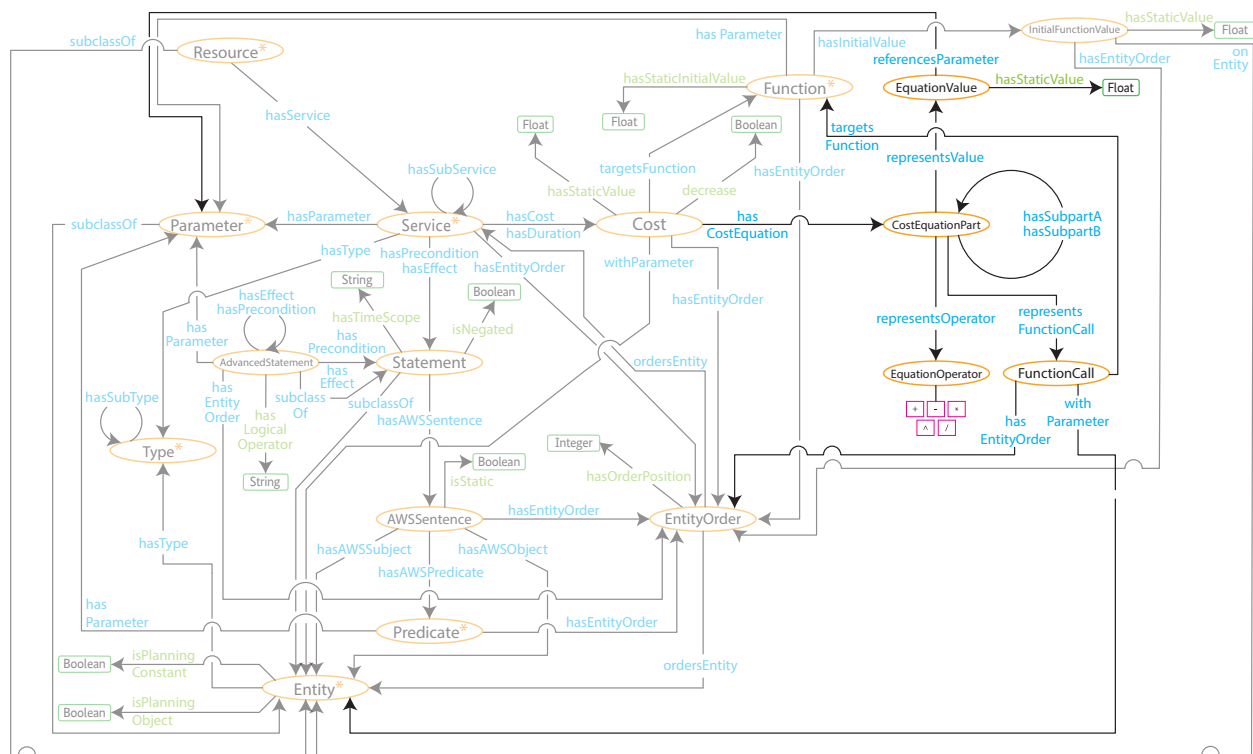
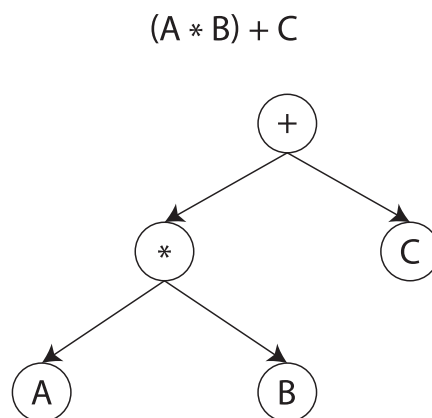


Figure 3.11: The cost equation concept

Figure 3.12: A binary expression tree expressing the formula  $(A * B) + C$

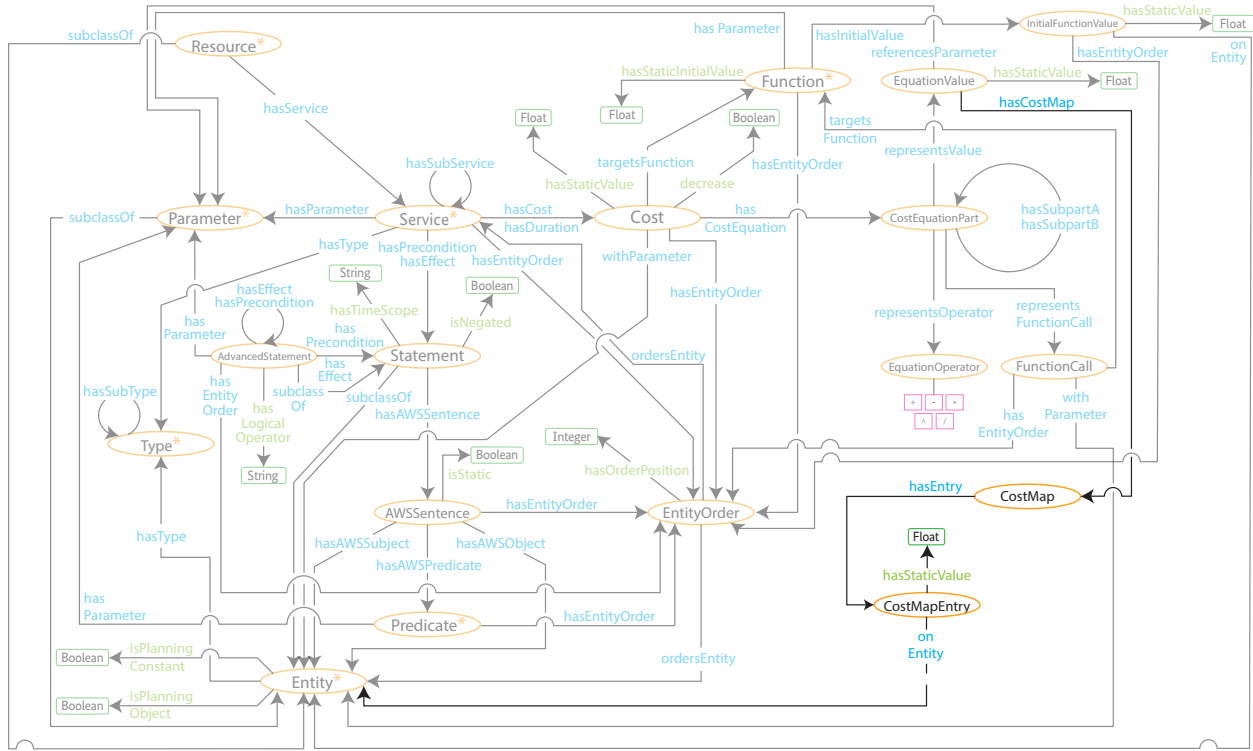


Figure 3.13: Parameter referencing in cost equations

A node in a binary expression tree that represents a Plato cost equation is modeled as an individual of the ‘CostEquationPart’ class. Each node can have child nodes connected through the ‘hasSubpartA’ and ‘hasSubpartB’ object properties. These child nodes must be distinguished because not all mathematical operators are commutative. For example,  $A / B$  is not equal to  $B / A$ . Each node can represent either a value, an operator, or a function call. This is expressed using one of the object properties ‘representsValue’, ‘representsOperator’, or ‘representsFunctionCall’.

To specify an operator, the equation part is associated with a predefined individual of the ‘EquationOperator’ class. To define a function call, an individual of the ‘FunctionCall’ class must be created. Each function call must reference the function being called through the ‘targetsFunction’ object property. If the function requires parameters, the values used in the function call must also be specified through the ‘withParameter’ object property. If more than one parameter is defined, the parameters must be ordered using the ‘EntityOrder’ concept introduced in Section 3.5.

To add a value to the expression, the cost equation part must be related to an individual of the ‘EquationValue’ class using the ‘representsValue’ object property. The value can either be a static value specified through the ‘hasStaticValue’ data property or the value of a parameter referenced through the ‘referencesParameter’ object property. If the parameter is non-numeric, a ‘CostMap’ must be provided through the ‘hasCostMap’ object property to map the possible parameter values to numeric values so that they can be used in the equation. Figure 3.13 shows this part of the Plato schema.

A ‘CostMap’ consists of multiple ‘CostMapEntry’ individuals, one for each possible value of a parameter. Each entry is associated with the cost map through the ‘hasEntry’ object property. In addition, it must provide a float value through the ‘hasStaticValue’ data property and point to the parameter value it represents through the ‘onEntity’ object property.

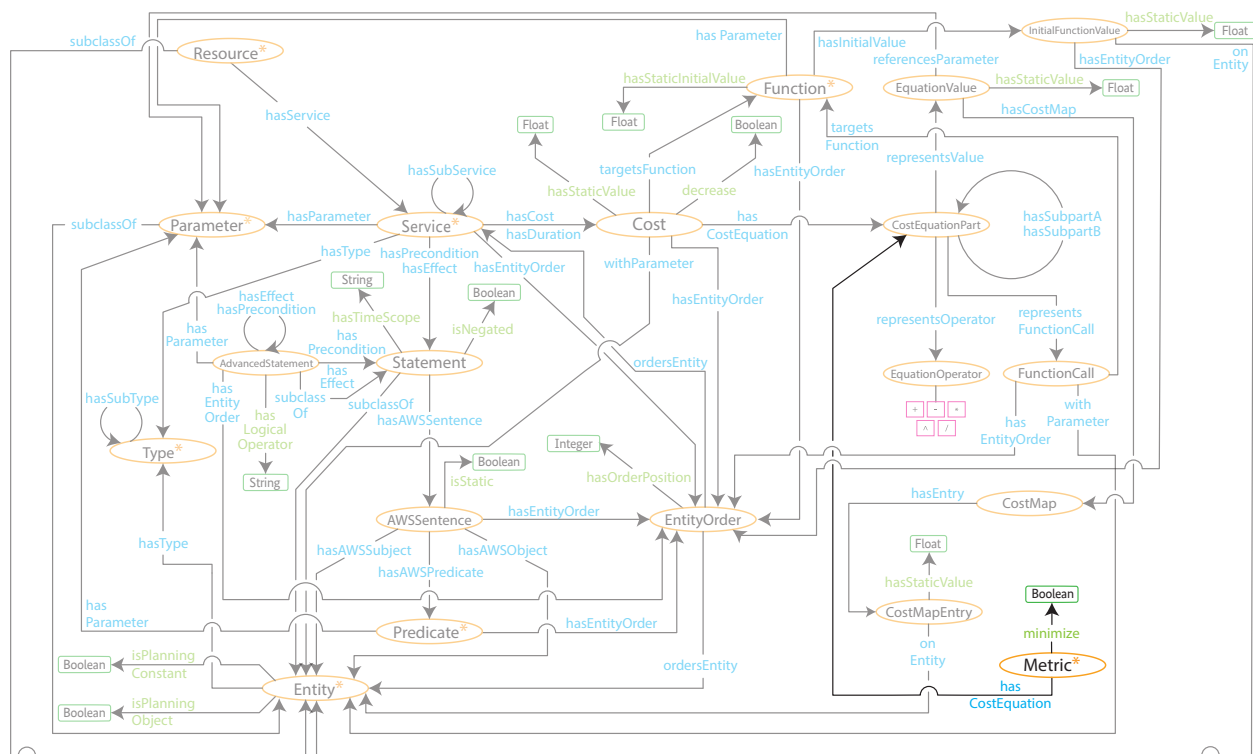


Figure 3.14: Cost metrics

### 3.9 Metrics

In many planning systems, the optimization behavior of the planner can be adapted to the current objective. For example, it may be desirable to minimize time instead of material costs. For this purpose, Plato contains the ‘Metric’ class, which can be used to control the optimization performed by the planning system. Figure 3.14 shows this part of the Plato schema.

The ‘Metric’ class can be used to specify which cost functions influence optimization. Each metric individual must reference a cost equation through the ‘hasCostEquation’ object property, and it must also have a planning name. In addition, it must be specified whether the metric is to be minimized or maximized by using the boolean ‘minimize’ data property, where `minimize == false` means maximize.

The cost equation referenced by a metric must not reference service parameters, since the metric is not associated with any individual service. Instead, the equation should reference cost functions using the ‘FunctionCall’ class. In addition, function calls may be multiplied by values in order to assign weights to each function.

### 3.10 Numeric Constants

The final concept covered by Plato concerns numeric values. Continuous numeric values are problematic in AI planning because they make the search space effectively infinite. As a result, most planning systems do not support continuous numeric values. While Plato allows continuous numeric values, it also offers a hybrid approach by allowing bounded continuous numeric values together with information about how these values should be discretized so that they can be used with most planning systems. Figure 3.15 shows this part of the Plato schema.

As mentioned above, numeric values are represented using the ‘Entity’ class or, more precisely, a subclass of ‘Entity’. To create a numeric constant, an individual of the ‘NumericConstant’ class must



represented in Plato into various formats, including PDDL planning domains and Asset Administration shells. For more information, refer to the documentation of KTF provided in its GitHub repository<sup>3</sup>.

## 5. Appendix

Table 5.1: All classes of the Plato scheme

| Class                          | Description                                                                                                                                                                                                                  |
|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| plato:AWSSentence              | A statement (Atomic World State Sentence) describing an atomic state                                                                                                                                                         |
| plato:AdvancedStatement        | An advanced statement used to describe disjunctive, universal, or existential preconditions.                                                                                                                                 |
| plato:Cost                     | Used to represent Costs that arise when executing a service                                                                                                                                                                  |
| plato:CostEquationPart         | A part of a cost equation, e.g. an operator, value or function call                                                                                                                                                          |
| plato:CostMap                  | Used to map parameter values to numeric values so that non numeric parameters can effect a cost equation                                                                                                                     |
| plato:CostMapEntry             | An Entry to a plato:CostMap                                                                                                                                                                                                  |
| plato:Entity                   | A superclass for all entities in the ontology to make modeling easier   <b>Sub-Classes:</b> plato:Resource, plato:Parameter, plato:NumericConstant, plato:NumericConstantReference, plato:Statement, plato:AdvancedStatement |
| plato:EntityOrder              | Used to order entities, e.g. service parameters                                                                                                                                                                              |
| plato:EquationOperator         | An Operator in a cost equation, e.g. '+'                                                                                                                                                                                     |
| plato:EquationValue            | A value in an equation                                                                                                                                                                                                       |
| plato:Function                 | A numerical function used to store numerical values, i.e. a numeric fluent in PDDL                                                                                                                                           |
| plato:FunctionCall             | Used to represent function calls in cost equations                                                                                                                                                                           |
| plato:InitialFunctionValue     | Used to store initial values for a function that has parameters and therefore can have more than one initial value                                                                                                           |
| plato:Metric                   | Used to represent a planning metric by providing a cost equation that, for example, weights cost functions                                                                                                                   |
| plato:NumericConstant          | A numeric constant that is used to convert continuous parameters (e.g. hole diameter) into discrete parameters                                                                                                               |
| plato:NumericConstantReference | Used to reference a single value or a value range of a numeric constant                                                                                                                                                      |
| plato:Parameter                | A parameter for a plato:Service (e.g. diameter of the hole that should be milled)                                                                                                                                            |
| plato:Predicate                | A predicate used for describing an atomic world state (e.g. 'at' - workpiece at milling machine)                                                                                                                             |
| plato:Resource                 | A machine in the production facility (e.g. a milling machine)                                                                                                                                                                |
| plato:Service                  | A service offered by a plato:Resource (e.g. milling a hole into a workpiece)                                                                                                                                                 |
| plato:Statement                | A statement describing a precondition or an effect   <b>Sub-Classes:</b> plato:AdvancedStatement                                                                                                                             |
| plato:Type                     | A type, e.g. service type, machine type etc.                                                                                                                                                                                 |

Table 5.2: All object properties of the Plato scheme

| Object Property                 | Description                                                                                                                         | Domain                                                                                                                              | Range                                    | Inverse            |
|---------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|--------------------|
| plato:hasAWSObject              | References an object of the AWSSentence                                                                                             | plato:AWSSentence                                                                                                                   | plato:Entity                             |                    |
| plato:hasAWSPredicate           | References the predicate of an AWSSentence                                                                                          | plato:AWSSentence                                                                                                                   | plato:Predicate                          |                    |
| plato:hasAWSSentence            | States that a precondition or an effect has an AWSSentence                                                                          | plato:Statement                                                                                                                     | plato:AWSSentence                        |                    |
| plato:hasAWSSubject             | References the subject of an AWSSentence                                                                                            | plato:Service                                                                                                                       | plato:Entity                             |                    |
| plato:hasCost                   | States that the execution of a service leads to some costs.                                                                         | plato:Service                                                                                                                       | plato:Cost                               |                    |
| plato:hasCostEquation           | States that a cost instance has an equation that determines the specific cost value. The cost instance can still have a static cost | plato:Cost, plato:Metric                                                                                                            | plato:CostEquationPart                   |                    |
| plato:hasCostMap                | Used for a plato:EquationValue that is determined by a non numeric parameter to map the parameter values to numeric values          | plato:EquationValue                                                                                                                 | plato:CostMap                            |                    |
| plato:hasDuration               | States that the execution of a service has a certain lead time.                                                                     | plato:Service                                                                                                                       | plato:Cost                               |                    |
| plato:hasEffect                 | States that a service has an effect that changes the world state when the service is executed                                       | plato:Service                                                                                                                       | plato:Statement                          |                    |
| plato:hasEntityOrder            | States that an instance has a group of ordered objectProperties relating to entities                                                | plato:Service, plato:AWSSentences, plato:Function, plato:InitialFunctionValue, plato:Predicate, plato:Cost, plato:AdvancedStatement | plato:EntityOrder                        |                    |
| plato:hasEntry                  | Used to add an entry to a plato:CostMap                                                                                             | plato:CostMap                                                                                                                       | plato:CostMapEntry                       |                    |
| plato:hasInitialValue           | States that a function has a initial value for a specific set of parameter values                                                   | plato:Function                                                                                                                      | plato:InitialFunctionValue               |                    |
| plato:hasParameter              | States that a service, a predicate, or a function has a parameter                                                                   | plato:Service, plato:Predicate, plato:Function                                                                                      | plato:Parameter                          |                    |
| plato:hasPrecondition           | States that a service has a precondition that must be satisfied before the service can be executed                                  | plato:Service, plato:AdvancedStatement                                                                                              | plato:Statement, plato:AdvancedStatement |                    |
| plato:hasService                | Used to state that a machine offers a service that it can execute                                                                   | plato:Resource                                                                                                                      | plato:Service                            | plato:isProvidedBy |
| plato:hasSubService             | States that a service has a another service that is its subtask                                                                     | plato:Service                                                                                                                       | plato:Service                            |                    |
| plato:hasSubType                | States that a type has a direct sub-type                                                                                            | plato:Type                                                                                                                          | plato:Type                               |                    |
| plato:hasSubpartA               | Used to reference the first child node of a Binary Expression Tree node describing an equation                                      | plato:CostEquationPart                                                                                                              | plato:CostEquationPart                   |                    |
| plato:hasSubpartB               | Used to reference the second child node of a Binary Expression Tree node describing an equation                                     | plato:CostEquationPart                                                                                                              | plato:CostEquationPart                   |                    |
| plato:hasType                   | States that a service has a type                                                                                                    | plato:Service, plato:Entity                                                                                                         | plato:Type                               |                    |
| plato:isAffectedBy              | States that a precondition or effect binding is affected by a parameter                                                             | plato:Binding                                                                                                                       | plato:Parameter                          |                    |
| plato:isProvidedBy              | Used to state that a service is offered by a specific machine                                                                       | plato:Service                                                                                                                       | plato:Resource                           |                    |
| plato:isEntry                   | References a CostMapEntry or an InitialFunctionValue with an Entity                                                                 | plato:CostMapEntry, plato:InitialFunctionValue                                                                                      | plato:Entity                             |                    |
| plato:isEntryOrder              | References an entry or service (for subtask ordering) that is ordered by this instance of EntityOrder                               | plato:EntityOrder                                                                                                                   | plato:Entity, plato:Service              |                    |
| plato:referencesNumericConstant | Used to reference a NumericConstant instance from a NumericConstantReference                                                        | plato:NumericConstantReference                                                                                                      | plato:NumericConstant                    |                    |
| plato:referencesParameter       | States that an EquationValue is determined by a parameter                                                                           | plato:EquationValue                                                                                                                 | plato:Parameter                          |                    |
| plato:representsFunctionCall    | States that a CostEquationPart is a function call.                                                                                  | plato:CostEquationPart                                                                                                              | plato:FunctionCall                       |                    |
| plato:representsOperator        | States that a CostEquationPart is an operator.                                                                                      | plato:CostEquationPart                                                                                                              | plato:EquationOperator                   |                    |
| plato:representsValue           | States that a CostEquationPart is a value.                                                                                          | plato:CostEquationPart                                                                                                              | plato:EquationValue                      |                    |
| plato:targetsFunction           | States that some cost should be applied to a specific cost function                                                                 | plato:Cost, plato:FunctionCall                                                                                                      | plato:Function                           |                    |
| plato:withParameter             | Used to specify parameters for a function call or the target function of a service cost                                             | plato:FunctionCall, plato:Cost                                                                                                      | plato:Entity                             |                    |

Table 5.3: All data properties of the Plato scheme

| Data Property               | Description                                                                                                      | Domain                                                                          | Range       |
|-----------------------------|------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------------|
| plato:decrease              | States that a cost should be applied to a cost function by decreasing it                                         | plato:Cost                                                                      | xsd:boolean |
| plato:hasDiscreteStepValue  | Used to set the step value for discretizing a numeric constant                                                   | plato:NumericConstant                                                           | xsd:float   |
| plato:hasLogicalOperator    | States that an advanced statement uses a specific logical operator to combine other statements.                  | plato:AdvancedStatement                                                         | xsd:string  |
| plato:hasMaximumValue       | Used to set a maximum value for a NumericConstant or a NumericConstantReference                                  | plato:NumericConstant, plato:NumericConstantReference                           | xsd:float   |
| plato:hasMinimumValue       | Used to set a minimum value for a NumericConstant or a NumericConstantReference                                  | plato:NumericConstant, plato:NumericConstantReference                           | xsd:float   |
| plato:hasOperatorValue      | Defines which operator an EquationOperator instance resembles                                                    | plato:EquationOperator                                                          | xsd:string  |
| plato:hasOrderPosition      | States the index of the entity ordered by a EntityOrder instance                                                 | plato:EntityOrder                                                               | xsd:integer |
| plato:hasStaticInitialValue | States that a function without parameters has a static initial value                                             | plato:Function                                                                  | xsd:float   |
| plato:hasStaticValue        | States that an instance has a static numeric value associated with it                                            | plato:Cost, plato:EquationValue, plato:CostMapEntry, plato:InitialFunctionValue | xsd:float   |
| plato:hasTimeScope          | Used to set a time scope (i.e. 'at start', 'at end', 'over all') for a precondition or an effect                 | plato:Statement                                                                 | xsd:string  |
| plato:isNegated             | Used to state whether the AWSSentence of a Statement is negated or not                                           | plato:Statement                                                                 | xsd:boolean |
| plato:isPlanningConstant    | Used to state that an entity is a planning constant                                                              | plato:Entity                                                                    | xsd:boolean |
| plato:isPlanningObject      | Used to state that an entity is a planning object                                                                | plato:Entity                                                                    | xsd:boolean |
| plato:isStatic              | Used to state that a AWSSentence is not a precondition or effect but rather a general statement about the domain | plato:AWSSentence                                                               | xsd:boolean |
| plato:minimize              | States whether the metric should be minimized or maximized                                                       | plato:Metric                                                                    | xsd:boolean |
| plato:referencesValue       | States that a NumericConstantReference is directed at a single value and not a value range                       | plato:NumericConstantReference                                                  | xsd:boolean |

<sup>3</sup><https://github.com/DavidJilg/KTF>

Table 5.4: All annotation properties of the Plato scheme

| Annotation Property   | Description                                                                  | Range      |
|-----------------------|------------------------------------------------------------------------------|------------|
| plato:hasPlanningName | Used to set a name that is used to represent the object in a planning domain | xsd:string |

Table 5.5: All individuals of the Plato scheme

| Individual                   | Description                      | Class                  |
|------------------------------|----------------------------------|------------------------|
| plato:AdditionOperator       | The '+' operator                 | plato:EquationOperator |
| plato:Boolean                | A boolean value                  | plato:Type             |
| plato:DivisionOperator       | The '/' operator                 | plato:EquationOperator |
| plato:ExponentialOperator    | The 'pow' (exponential) operator | plato:EquationOperator |
| plato:False                  | The Boolean Value of 'False'     | plato:Entity           |
| plato:MultiplicationOperator | The '*' operator                 | plato:EquationOperator |
| plato:SubtractionOperator    | The '-' operator                 | plato:EquationOperator |
| plato:True                   | The Boolean Value of 'True'      | plato:Entity           |

# References

- [1] Klein, P., Malburg, L., Bergmann, R.: Ftonto: A domain ontology for a fischertechnik simulation production factory by reusing existing ontologies. In: Lernen, Wissen, Daten, Analysen (2019), <https://api.semanticscholar.org/CorpusID:202760112>
- [2] Lassila, O., Swick, R.R.: Resource Description Framework (RDF) Model and Syntax Specification (1999), <http://www.w3.org/TR/1999/REC-rdf-syntax-19990222/>
- [3] Malburg, L., Klein, P., Bergmann, R.: Converting semantic web services into formal planning domain descriptions to enable manufacturing process planning and scheduling in industry 4.0. Eng. Appl. Artif. Intell. **126**, 106727 (2023), <https://api.semanticscholar.org/CorpusID:260010641>